

Caesar Cipher and Playfair Cipher Implementation

Course Title: Information Security Laboratory

Course No: 0714 02 CSE 4108

Submitted to,

Dr. Anupam Kumar Bairagi

Professor,

Computer Science and Engineering Discipline,

Khulna University, Khulna-9208

Submitted by,

Md. Siam Ahmed

Student ID: 230202

Year: 4th, Term: 1st

Computer Science and Engineering Discipline,

Khulna University, Khulna-9208

Caesar Cipher:

Encryption:

```
#include <iostream>
#include <string>
using namespace std;

string caesarEncrypt(string &plaintext)
{
    string caesar_cipher;

    for (char character : plaintext)
    {
        char cipher_character;
        if (character < 'A' || character > 'z' || (character > 'Z' && character <
'a'))
            cipher_character = character;
        else if (character < 'a')
        {
            cipher_character = (((character - 'A' + 3) % 26) + 'A')^32;
        }
        else
        {
            cipher_character = (((character - 'a' + 3) % 26) + 'a')^32;
        }

        caesar_cipher.push_back(cipher_character);
    }
    return caesar_cipher;
}

int main()
{
    cout<<endl<<"Caesar Cipher Encryption:-\n";
    string plaintext;
    cout << "Enter the Plaintext: ";
    getline(cin, plaintext);

    string caesar_cipher = caesarEncrypt(plaintext);

    cout << "Encrypted Caesar Cipher: " << caesar_cipher << endl;

    return 0;
}
```

Input and Output:

Caesar Cipher Encryption:-

Enter the Plaintext: Life is like riding a bicycle. To keep your balance, you must keep moving.

Encrypted Caesar Cipher: oLIH LV OLNH ULGLQJ D ELFBFOH. wR NHHS BRXU EDODQFH, BRX PXWV NHHS PRYLQJ.

Decryption:

```
#include <iostream>
#include <string>
using namespace std;

string caesarDecrypt(string &caesar_cipher)
{
    string plaintext;

    for (char cipher_character : caesar_cipher)
    {
        char character;
        if (cipher_character < 'A' || cipher_character > 'z' || (cipher_character > 'Z' && cipher_character < 'a'))
            character = cipher_character;
        else if (cipher_character < 'a')
        {
            character = (((cipher_character - 'A' - 3+26) % 26) + 'A')^32;
        }
        else
        {
            character = (((cipher_character - 'a' - 3+26) % 26) + 'a')^32;
        }

        plaintext.push_back(character);
    }
    return plaintext;
}
```

```

int main()
{
    cout<<endl<<"Caesar Cipher Decryption:-\n";
    string caesar_cipher;
    cout << "Enter the Caesar Cipher: ";
    getline(cin, caesar_cipher);

    string plaintext = caesarDecrypt(caesar_cipher);

    cout << "Decrypted Plaintext: " << plaintext << endl;

    return 0;
}

```

Input and Output:

```

Caesar Cipher Decryption:-
Enter the Caesar Cipher: oLIH LV OLNH ULGLQJ D ELFBFOH. wR NHHS BRXU EDODQFH, BRX PXVW NHHS PRYLQJ.
Decrypted Plaintext: Life is like riding a bicycle. To keep your balance, you must keep moving.

```

Playfair Cipher:

Encryption:

```
#include <iostream>
#include <string>
#include <vector>
#include <map>
using namespace std;

string playfairEncrypt(string &keyword, string &plaintext)
{
    string temp_keyword, temp_plaintext;
    for (char character : keyword)
    {
        if ((character >= 'A' && character <= 'Z') || (character >= 'a' &&
character <= 'z'))
            temp_keyword.push_back(character);
    }
    for (char character : plaintext)
    {
        if ((character >= 'A' && character <= 'Z') || (character >= 'a' &&
character <= 'z'))
            temp_plaintext.push_back(character);
    }

    string caesar_cipher;

    vector<vector<char>> square_matrix(5, vector<char>(5));
    int r = 0, c = 0;
    map<char, pair<int, int>> mp;

    for (char character : keyword)
    {
        if (character > 'Z')
            character ^= 32;
        if (mp.find(character) == mp.end())
        {
            square_matrix[r][c] = character;
```

```

        if (character == 'I' || character == 'J')
        {
            mp['I'] = {r, c};
            mp['J'] = {r, c};
        }
        else
            mp[character] = {r, c};
        c++;
        if (c == 5)
        {
            r++;
            c = 0;
        }
    }
}

for (char character = 'A'; character <= 'Z'; character += 1)
{

    if (mp.find(character) == mp.end())
    {
        square_matrix[r][c] = character;
        if (character == 'I' || character == 'J')
        {
            square_matrix[r][c] = 'I';
            mp['I'] = {r, c};
            mp['J'] = {r, c};
        }
        else
            mp[character] = {r, c};
        c++;
        if (c == 5)
        {
            r++;
            c = 0;
        }
    }
}

```

```

"The square matrix from the given keyword: \n";
for (int i = 0; i < 5; i++)
{
    for (int j = 0; j < 5; j++)
        if (square_matrix[i][j] == 'I' || square_matrix[i][j] == 'J')
            cout << "IJ ";
        else
            cout << square_matrix[i][j] << ' ';
    cout << endl;
}

string message;
int i;
for (i = 1; i < plaintext.size(); i++)
{
    if (plaintext[i] < 'a')
        plaintext[i] ^= 32;
    message.push_back(plaintext[i - 1]);
    if (plaintext[i] == plaintext[i - 1])
    {
        if (plaintext[i] == 'x')
        {
            message.push_back('q');
        }
        else
            message.push_back('x');
    }
    else
    {
        message.push_back(plaintext[i]);
        i++;
    }
}
if (i == plaintext.size())
{
    message.push_back(plaintext[i - 1]);
    if (plaintext[i - 1] == 'x')
    {
        message.push_back('q');
    }
    else
        message.push_back('x');
}

cout << "Message: ";

```

```

for (i = 0; i < message.size(); i += 2)
{
    cout << message[i] << message[i + 1] << " ";
}
cout << endl;
for (i = 0; i < message.size(); i += 2)
{
    if (mp[(message[i] ^ 32)].first == mp[(message[i + 1] ^ 32)].first)
    {
        r = mp[(message[i] ^ 32)].first;
        int c1 = mp[(message[i] ^ 32)].second;
        int c2 = mp[(message[i + 1] ^ 32)].second;
        c1 = (c1 + 1) % 5;
        c2 = (c2 + 1) % 5;
        caesar_cipher.push_back(square_matrix[r][c1]);
        caesar_cipher.push_back(square_matrix[r][c2]);
    }
    else if (mp[(message[i] ^ 32)].second == mp[(message[i + 1] ^
32)].second)
    {
        c = mp[(message[i] ^ 32)].second;
        int r1 = mp[(message[i] ^ 32)].first;
        int r2 = mp[(message[i + 1] ^ 32)].first;
        r1 = (r1 + 1) % 5;
        r2 = (r2 + 1) % 5;
        caesar_cipher.push_back(square_matrix[r1][c]);
        caesar_cipher.push_back(square_matrix[r2][c]);
    }
    else
    {
        int r1 = mp[(message[i] ^ 32)].first;
        int r2 = mp[(message[i + 1] ^ 32)].first;
        int c1 = mp[(message[i] ^ 32)].second;
        int c2 = mp[(message[i + 1] ^ 32)].second;

        caesar_cipher.push_back(square_matrix[r1][c2]);
        caesar_cipher.push_back(square_matrix[r2][c1]);
    }
}
return caesar_cipher;
}

```



```

int main()
{
    cout << endl
        << "Playfair Cipher Encryption:-\n";
    string keyword;
    cout << "Enter the Keyword: ";
    getline(cin, keyword);
    string plaintext;
    cout << "Enter the Plaintext: ";
    getline(cin, plaintext);

    string playfair_cipher = playfairEncrypt(keyword, plaintext);

    cout << "Ciphertext: ";

    for (int i = 0; i < playfair_cipher.size(); i += 2)
    {
        cout << playfair_cipher[i] << playfair_cipher[i + 1] << " ";
    }
    cout << endl;

    return 0;
}

```

Input and Output:

```

Playfair Cipher Encryption:-
Enter the Keyword: Hijacking
Enter the Plaintext: thebankasia
H I J A C K
N G B D E
F L M O P
Q R S T U
V W X Y Z
Message: th eb an ka si ax
Ciphertext: QC ND HB HC RA BA

```

Decryption:

```
#include <iostream>

#include <string>
#include <vector>
#include <map>
using namespace std;

string playfairEncrypt(string &keyword, string &ciphertext)
{
    string temp_keyword, temp_ciphertext;
    for (char character : keyword)
    {
        if ((character >= 'A' && character <= 'Z') || (character >= 'a' &&
character <= 'z'))
            temp_keyword.push_back(character);
    }
    for (char character : ciphertext)
    {
        if ((character >= 'A' && character <= 'Z') || (character >= 'a' &&
character <= 'z'))
            temp_ciphertext.push_back(character);
    }
    keyword = temp_keyword;
    ciphertext = temp_ciphertext;

    vector<vector<char>> square_matrix(5, vector<char>(5));
    int r = 0, c = 0;
    map<char, pair<int, int>> mp;

    for (char character : keyword)
    {
        if (character > 'Z')
            character ^= 32;
        if (mp.find(character) == mp.end())
        {
            square_matrix[r][c] = character;
```

```

        if (character == 'I' || character == 'J')
        {
            mp['I'] = {r, c};
            mp['J'] = {r, c};
        }
        else
            mp[character] = {r, c};
        c++;
        if (c == 5)
        {
            r++;
            c = 0;
        }
    }
}

for (char character = 'A'; character <= 'Z'; character += 1)
{
    if (mp.find(character) == mp.end())
    {
        square_matrix[r][c] = character;
        if (character == 'I' || character == 'J')
        {
            square_matrix[r][c] = 'I';
            mp['I'] = {r, c};
            mp['J'] = {r, c};
        }
        else
            mp[character] = {r, c};
        c++;
        if (c == 5)
        {
            r++;
            c = 0;
        }
    }
}

```

```

"The square matrix from the given keyword: \n";
for (int i = 0; i < 5; i++)
{
    for (int j = 0; j < 5; j++)
        if (square_matrix[i][j] == 'I' || square_matrix[i][j] == 'J')
            cout << "IJ ";
        else
            cout << square_matrix[i][j] << ' ';
    cout << endl;
}

int i;

string message;
for (i = 0; i < ciphertext.size(); i += 2)
{
    if (ciphertext[i] > 'Z')
        ciphertext[i] ^= 32;
    if (ciphertext[i + 1] > 'Z')
        ciphertext[i + 1] ^= 32;

    if (mp[ciphertext[i]].first == mp[ciphertext[i + 1]].first)
    {
        r = mp[ciphertext[i]].first;
        int c1 = mp[ciphertext[i]].second;
        int c2 = mp[ciphertext[i + 1]].second;
        c1 = (c1 - 1 + 5) % 5;
        c2 = (c2 - 1 + 5) % 5;
        message.push_back(square_matrix[r][c1]);
        message.push_back(square_matrix[r][c2]);
    }
    else if (mp[ciphertext[i]].second == mp[ciphertext[i + 1]].second)
    {
        c = mp[ciphertext[i]].second;
        int r1 = mp[ciphertext[i]].first;
        int r2 = mp[ciphertext[i + 1]].first;
        r1 = (r1 - 1 + 5) % 5;
        r2 = (r2 - 1 + 5) % 5;
        message.push_back(square_matrix[r1][c]);
        message.push_back(square_matrix[r2][c]);
    }
}

```

```

        else
        {
            int r1 = mp[ciphertext[i]].first;
            int r2 = mp[ciphertext[i + 1]].first;
            int c1 = mp[ciphertext[i]].second;
            int c2 = mp[ciphertext[i + 1]].second;

            message.push_back(square_matrix[r1][c2]);
            message.push_back(square_matrix[r2][c1]);
        }
    }
    cout << "Message: ";
    for (i = 0; i < message.size(); i += 2)
    {
        if (message[i] < 'a')
            message[i] ^= 32;
        if (message[i + 1] < 'a')
            message[i + 1] ^= 32;
        cout << message[i] << message[i + 1] << " ";
    }
    cout << endl;

    return message;
}

int main()
{
    cout << endl
        << "Playfair Cipher Decryption:-\n";
    string keyword;
    cout << "Enter the Keyword: ";
    getline(cin, keyword);
    string ciphertext;
    cout << "Enter the ciphertext: ";
    getline(cin, ciphertext);

    string plaintext = playfairEncrypt(keyword, ciphertext);

    cout << "Original Plaintext: "<<plaintext<<endl;

    return 0;
}

```

Input and Output:

```
Playfair Cipher Decryption:-  
Enter the Keyword: Hijacking  
Enter the ciphertext: QC ND HB HC RA BA  
H I J A C K  
N G B D E  
F L M O P  
Q R S T U  
V W X Y Z  
Message: th eb an ka si ax  
Original Plaintext: thebankasi ax
```